Project Model Selection

Tiny code (SQL) - Fine-Tuning Lightweight Language Models for BigQuery SQL Generation

Sanjay Kumar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SanjayKumar@sandiego.edu

Santa Das Karmakar
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SDasKarmakar@sandiego.edu

Sethuraman Krishnasamy
Master of Science in Applied Artificial Intelligence
Shiley Marcos School of Engineering / University of San Diego
SKrishnasamy@sandiego.edu

Project Model Selection (AI590)

Method Selection and Justification: Fine-Tuning Gemma-3-4b-it with LoRA for BigQuery Text-to-SQL Generation

After exploring the Gretel synthetic text-to-SQL dataset (50k training examples + 5k test) and confirming the project goal — building an **expert BigQuery developer assistant** that converts natural-language requests into correct, production-ready GoogleSQL — we evaluated multiple approaches and selected **LoRA fine-tuning of the Gemma-3-4b-it large language model** (via Hugging Face + PyTorch/TRL) as the core method. This choice directly satisfies all program requirements while being practical, effective, and aligned with current industry and academic best practices.

Why a Deep-Learning / Neural-Network Model (Transformer) Is Required and Superior

Text-to-SQL is fundamentally a **sequence-to-sequence generation task** that requires deep understanding of:

- Database schema context
- User intent
- SQL syntax
- BigQuery-specific dialects (SAFE_CAST, backticks, QUALIFY, etc.)

Traditional machine-learning methods (e.g., rule-based parsers, Decision Trees, or even older seq2seq models like T5) struggle with variable-length output, complex JOINs, window functions, and schema grounding. They cannot reliably produce executable SQL at scale. In contrast, **Transformer-based large language models** (decoder-only architecture) excel at this because they have already learned code and reasoning patterns during pre-training. The project guidelines explicitly mandate at least one neural-network/deep-learning model, and a modern LLM satisfies this while delivering far higher performance.

Why Fine-Tuning a Pre-Trained Model (Gemma-3-4b-it) via Hugging Face Instead of Training from Scratch

- **Efficiency and feasibility**: Training a 4B-parameter model from scratch would require massive data and compute (weeks on multiple GPUs). Fine-tuning reuses Gemma-3's existing knowledge of code, SQL, and instruction following, needing only a few epochs on our 50k synthetic examples.
- **Meets program requirements**: Explicitly allowed (and encouraged) via libraries like Hugging Face, PyTorch, or TRL.
- **Model choice rationale**:
    - **Gemma-3-4b-it** (instruction-tuned variant) is Google's own open model, optimized for chat-style tasks exactly like "user request → SQL + explanation".
    - 4B size strikes the best balance: powerful enough for high-quality SQL (outperforms smaller models) yet small enough to fine-tune on a **single L4 GPU** (the exact hardware our GKE node pool provides).
    - Instruction-tuned ("-it") version already understands the chat template we use (<|begin_of_text|> … <|eot_id|>).

Why LoRA (Low-Rank Adaptation) Specifically

LoRA is a **parameter-efficient fine-tuning (PEFT)** technique that freezes the base model and only trains small adapter matrices (rank r=32 in our config). This:

- Dramatically reduces memory and training time (fits in 44 GiB GPU memory with batch_size=4 and gradient accumulation).
- Maintains near full-fine-tune performance.

- Is the de-facto standard in 2025–2026 for production text-to-SQL projects.

Our config.yaml (lora r=32, alpha=64, target_modules on all attention + MLP layers, paged_adamw_32bit optimizer) follows exactly the recommendations in Google's official documentation.

Evidence That This Is How Others Solve Similar Problems

This approach is **not novel** (as permitted) — it mirrors widespread practice:

- Google's own official guide (Feb 2026) is titled "Fine-Tune Gemma using Hugging Face Transformers and QLoRA" and uses **the exact same dataset family** (Gretel synthetic text-to-SQL) for a natural-language-to-SQL use case.
- Multiple public models on Hugging Face (e.g., litert-community/gemma3-1b-ft-text-to-sql, parole-study-viper/gemma3-text-to-sql, Yuk050/gemma-3-1b-text-to-sql-model) were created by LoRA-fine-tuning Gemma-3 variants on the Gretel dataset.
- Industry leaders follow the same pattern: Defog's SQLCoder family (fine-tuned specifically for text-to-SQL and often outperforming GPT-4 on SQL benchmarks), Rubrik's SQL Copilot (Llama-3 fine-tuned on Gretel data), and numerous open-source projects using LoRA/QLoRA + Gretel or Spider-style datasets.
- Recent surveys (e.g., "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL", 2025) confirm that **LLM fine-tuning (especially PEFT)** has become the dominant, highest-accuracy paradigm over older PLMs or pure in-context learning.

Resource and Practical Considerations

Full fine-tuning or larger models (e.g., 27B) would exceed our single-GPU constraint and training budget. LoRA + Gemma-3-4b + our custom preprocessing (BigQuery transpilation with sqlglot, negative sampling, value grounding hints, style augmentation, and <think> reasoning tags) keeps training to ~20–30 steps max while delivering production-grade BigQuery SQL. The entire pipeline (GKE cluster + GPU node pool + Artifact Registry Docker image + Kubernetes Job) was built to make this training reproducible and scalable — far beyond a simple Colab notebook.

Original Work Element

While we drew motivation from Google's Gemma fine-tuning guide and Gretel examples, our implementation is original:

- BigQuery-specific normalization and dialect enforcement
- Multi-layer data augmentation tailored for enterprise schemas
- Full cloud-native MLOps infrastructure (GKE, Workload Identity, GCS Fuse, secrets management)
- Custom Gemma-3 chat template with <think> tags for better chain-of-thought reasoning

In summary, **LoRA fine-tuning of Gemma-3-4b-it via Hugging Face** is the best method for our project because it:

- Satisfies every program requirement (deep learning + model training)
- Matches exactly how Google, Defog, and the broader community solve text-to-SQL today
- Is computationally realistic for our environment
- Produces a genuinely useful BigQuery assistant rather than a toy prototype

This approach delivers the highest accuracy with the lowest resource overhead — the optimal balance for an applied capstone project.